Connecting a Next.js + Supabase shop to Meta Ads

Catalog sync, tracking, and promo reels with an “Order Now” button. A step-by-step plan split between the developer and the business owner.

	Developer (you)	Business owner (client)
Owns	The website code, feed endpoint, tracking code, DNS record, testing.	The Meta accounts (portfolio, Page, Instagram, ad account, billing), creatives, ad budget.
Delivers	A product feed URL, a working Pixel + Conversions API, a tested checkout funnel.	Verified accounts, Pixel ID, catalog connected to the feed, reels, running ads.
Never does	Never owns the client's Meta accounts or asks for his password.	Never shares his Facebook password; gives access through Meta's partner / people settings.

How the pieces fit

Important constraints to know before starting

- **The “Order Now” button comes from the ad, not the organic reel.** Meta lists Order Now as a call-to-action option for single image/video ads on both Facebook and Instagram. In my research I found no way to add that button to an organic reel through the API. So the client posts the reel normally, then promotes it (or runs it as an ad) and picks Order Now in Ads Manager.
- **Catalog and Pixel work independently of Shops checkout.** Whether Instagram/Facebook Shops (in-app checkout, organic product tags) is available depends on the country and the account. The plan below sends buyers to your own website, which works regardless. The client should check Commerce Manager for his account's eligibility.
- **The Meta interface changes often.** Menu names below match current guides, but labels may differ slightly. Where a label is unclear, search the page for the keyword in *italics*.

Suggested timeline

When	Developer	Client
Day 1-2	D0-D2: decisions, data clean-up, feed endpoint built and deployed.	C0-C4: business portfolio, Page, Instagram, ad account, billing.
Day 2-3	D8: add the DNS TXT record the client sends (**start early: propagation can take up to 72 hours**).	C5-C6: create Pixel, add domain and copy the TXT value.
Day 3-5	D3, D5: test feed, install Pixel on the site.	C7-C8: create catalog, connect feed URL, share access, generate API token.
Day 5-8	D6-D7: Conversions API, Purchase event, de-duplication.	C9: shoot first reels.
Day 8-10	D10: full QA together, fix issues, D11 handover.	C10: launch first campaign with small budget.
Ongoing	Monitor errors, update feed as the catalog grows.	C11: weekly routine, creative refresh.

Contents

Contents . . . 2

Part 1: Developer tasks 3

D0. Decisions to settle with the client first . . . 3
D1. Prepare the Supabase data . . . 3
D2. Build the catalog feed endpoint . . . 4
D3. Hand the feed to the client and clear the errors . . . 5
D4. Make product pages ad-ready . . . 5
D5. Install the Meta Pixel in Next.js . . . 5
D6. Add the Conversions API (server-side events) . . . 6
D7. Checkout and the Purchase event . . . 7
D8. Domain verification: the DNS record . . . 8
D9. Consent, privacy and security . . . 8
D10. Testing and QA checklist . . . 8
D11. Handover, monitoring and optional real-time sync . . . 9

Part 2: Business owner tasks 10

C0. Before you start . . . 10
C1. Create the Meta Business portfolio . . . 10
C2. Create or connect your Facebook Page . . . 10
C3. Instagram professional account . . . 10
C4. Create the ad account and add billing . . . 11
C5. Create the Pixel (dataset) . . . 11
C6. Verify your domain . . . 11
C7. Create the catalog and connect the feed . . . 11
C8. Give the developer access (without sharing your password) . . . 12
C9. Create the reels (your creative work) . . . 12
C10. Launch an ad with the “Order Now” button . . . 13
C11. Weekly routine and good habits . . . 14

Appendix A. Handover sheet 15

Appendix B. Troubleshooting 15

Appendix C. References 16

PART 1

What the developer does

Website, data, feed, tracking, DNS and testing

Part 1: Developer tasks

Steps are in the order you should do them. Items marked **Needs from client** block you until he provides them, so send him the list early.

D0. Decisions to settle with the client first

These choices are painful to change later, because the product IDs in the feed must match the IDs sent by the Pixel.

Decision	Recommendation	Why it matters
Product ID (retailer id)	Use the Supabase **sku** (or a permanent UUID). Never reuse or rename it.	Feed *id* and Pixel *content_ids* must be identical, or catalog ads cannot match events to products.
Currency	One currency everywhere (DZD, unless he sells in EUR/USD). Format as *1500.00 DZD*.	Meta needs the price with an ISO currency code. Mixed currencies cause rejected items and wrong ROAS.
What counts as a “Purchase”	Discuss with client: see D7 (cash on delivery changes this).	Ads optimise toward whatever you call Purchase.
Variants (size/color)	Each variant = its own feed row with its own id, sharing an *item_group_id*.	Meta treats each row as a unique product variant.
Checkout destination	Ads send people to the product page (with UTM tags), not the home page.	Fewer clicks to order = higher conversion.
Cookie consent	Decide if Pixel loads before or after consent (see D9).	Legal and tracking-accuracy trade-off.

D1. Prepare the Supabase data

Goal: every product has the fields Meta requires, in clean form.

1 Open the Supabase table editor for **products** and confirm these columns exist (add them with a migration if not):

Column	Type	Feed field	Rules
sku	text, unique	id	Permanent. Max 100 characters.
name	text	title	Plain text, no ALL CAPS, no promo spam. Under ~150 characters.
description	text	description	Plain text, no HTML, at least one real sentence.
price	numeric	price	Regular price, number only. Sent as *1500.00 DZD*.
stock	integer	availability	> 0 sends *in stock*, otherwise *out of stock*.
slug	text, unique	link	Public product URL: https://yourshop.com/products/slug
image_url	text	image_link	Public HTTPS image, no login, at least 500x500 px (1080x1080 ideal).
brand	text	brand	Shop name if he sells his own products.
published	boolean	(filter)	Only published items go into the feed.
sale_price (optional)	numeric	sale_price	Only when a real discount is active.

Column	Type	Feed field	Rules
item_group_id (optional)	text	item_group_id	Shared by all variants of one product.

1. Check that every **image_url** opens in a private browser window (no sign-in) and is a direct image file. Supabase Storage public-bucket URLs work. Meta cannot fetch images behind authentication or hotlink protection.
2. Fix duplicates: run a query for repeated skus and for products with empty name, price or image.
3. Make sure the service-role key is only used on the server (never in a `NEXT_PUBLIC_` variable).

> **Done when:** A query on the products table returns zero rows with missing sku, name, price, slug or image_url where published = true.

D2. Build the catalog feed endpoint

Goal: a URL that returns your live catalog as CSV, which Meta fetches on a schedule (hourly, daily or weekly).

1. Create the environment variables: `META_FEED_TOKEN` (a long random string), `NEXT_PUBLIC_SITE_URL`, `SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`.
2. Create the route file below. The token in the query string keeps strangers from scraping the feed (Meta stores the full URL you give it, so the token goes along).
3. Deploy, then test: `curl 'https://yourshop.com/api/meta/feed.csv?token=...' | head`. Confirm the header row and a few products look right, and that a wrong token returns 403.
4. Open the CSV in a spreadsheet and sample 5 rows: price has a currency, links open the right page, image links load.

File: app/api/meta/feed.csv/route.ts

```
// app/api/meta/feed.csv/route.ts
import { createClient } from "@supabase/supabase-js";

export const dynamic = "force-dynamic";
const SITE = process.env.NEXT_PUBLIC_SITE_URL!;
const q = (v: unknown) => `"${String(v ?? "").replace(/"/g, '""')}"`;

export async function GET(req: Request) {
  const token = new URL(req.url).searchParams.get("token");
  if (token !== process.env.META_FEED_TOKEN)
    return new Response("Forbidden", { status: 403 });

  const supabase = createClient(process.env.SUPABASE_URL!,
                                process.env.SUPABASE_SERVICE_ROLE_KEY!);
  const { data, error } = await supabase
    .from("products")
    .select("sku,name,description,price,stock,slug,image_url,brand")
    .eq("published", true);
  if (error) return new Response(error.message, { status: 500 });

  const header = ["id","title","description","availability","condition",
                  "price","link","image_link","brand"].join(",");
  const rows = data.map((p) => [
    q(p.sku), q(p.name), q((p.description ?? "").slice(0, 4900)),
    p.stock > 0 ? "in stock" : "out of stock", "new",
    q(`${Number(p.price).toFixed(2)} DZD`),
    q(`${SITE}/products/${p.slug}`), q(p.image_url), q(p.brand ?? "YourBrand"),
  ].join(","));

  return new Response([header, ...rows].join("\n"), {
    headers: { "Content-Type": "text/csv; charset=utf-8", "Cache-Control": "no-store" },
  });
}
```

> **Note:** Meta accepts CSV, TSV, XML and Google Sheets feeds. A scheduled **replace** upload deletes items that are missing from the file, which is exactly what you want for an unpublished product. Keep an **update** schedule for the optional real-time sync in D11.

> **Done when:** The URL returns a valid CSV with the live products, and rejects requests without the token.

D3. Hand the feed to the client and clear the errors

Goal: Meta ingests the feed with no blocking errors.

1. Send the client the full feed URL (including token) through a password manager or another secure channel, not a public chat. He pastes it into Commerce Manager in step C7.
2. After the first upload, ask him for a screen share of **Commerce Manager > Catalog > Overview / Diagnostics**, or get catalog-level access (C8) and look yourself.
3. Fix errors in the data or the endpoint, not in Meta. The usual ones: price missing a currency, image too small or not reachable, link not on the verified domain, description empty, availability invalid.
4. Ingestion is asynchronous, so wait for the next scheduled run, or ask the client to click *Upload now* in the feed settings, after each fix.

> **Needs from client:** The catalog must exist and the domain must be verified (C6-C7). Items from an unverified domain can be disapproved.

> **Done when:** Items show as active in Catalog > Items, and the diagnostics page shows no blocking errors.

D4. Make product pages ad-ready

1. **Stable URLs.** The product URL in the feed is the destination of the “Order Now” ad. Keep the slug permanent; if you change it, add a 301 redirect.
2. **One click to order.** From the product page the user should reach the order form in one step. Test on a real phone over mobile data. Most reel traffic is mobile.
3. **Speed.** Run Lighthouse on a product page. Aim for under about 3 seconds to interactive on mobile; compress images, use `next/image`.
4. **Preserve UTM parameters.** Ads will add `utm_source`, `utm_campaign` and Meta's click id `fbclid`. Do not strip them in redirects. Store them (and the `_fbc` cookie) with the order so you can tie sales to the ad.
5. **Open Graph tags** on product pages: `og:title, og:description, og:image, og:url` (use `generateMetadata`).
6. **Out-of-stock handling.** Keep the page live and show “out of stock”; the feed will flag it so ads stop showing the item.

D5. Install the Meta Pixel in Next.js

Goal: the browser reports PageView, ViewContent, AddToCart, InitiateCheckout and Purchase.

> **Needs from client:** The **Pixel ID** (created in step C5).

1. Add `NEXT_PUBLIC_META_PIXEL_ID` to your environment variables (local and hosting provider).
2. Create the component below and render it once in `app/layout.tsx`. Inside the template string paste the standard base code Meta shows in **Events Manager > your Pixel > Install > Install code manually** (the “!function(f,b,e,v,n,t,s)...” snippet), then keep the `fbq('init')` and first PageView lines.
3. Create the small helper in `lib/meta.ts` so every event goes through one function.
4. Fire events from the right places: see the table after the code.

File: components/MetaPixel.tsx

```
// components/MetaPixel.tsx
"use client";
import Script from "next/script";
import { usePathname } from "next/navigation";
import { useEffect, useRef } from "react";

const PIXEL_ID = process.env.NEXT_PUBLIC_META_PIXEL_ID!;

export default function MetaPixel() {
  const pathname = usePathname();
  const first = useRef(true);
  useEffect(() => {                      // PageView on client-side navigation
    if (first.current) { first.current = false; return; }
    (window as any).fbq?.("track", "PageView");
  }, [pathname]);

  return (
    <Script id="meta-pixel" strategy="afterInteractive">{`
      /* paste Meta's standard base code here */
      fbq('init', '${PIXEL_ID}');
      fbq('track', 'PageView');
    `}</Script>
  );
}
```

File: lib/meta.ts

```
// lib/meta.ts  (browser helper)
export const newEventId = () => crypto.randomUUID();

export function track(name: string, data: Record<string, unknown> = {}, eventId?: string) {
  (window as any).fbq?.("track", name, data, eventId ? { eventID: eventId } : undefined);
}

// usage on a product page
track("ViewContent", { content_ids: [p.sku], content_type: "product",
                       content_name: p.name, value: p.price, currency: "DZD" });
```

Event	Fire when	Parameters
PageView	Every page load and route change	(none)
ViewContent	Product page opens	content_ids:[sku], content_type:'product', content_name, value, currency
AddToCart	Add to cart / Order now clicked	content_ids:[sku], content_type:'product', value, currency
InitiateCheckout	Checkout / order form opens	content_ids, contents:[{id,quantity}], num_items, value, currency
Purchase	Order is placed (see D7)	content_ids, contents, content_type:'product', value, currency + eventID

Watch out: **content_ids must be the exact feed ids (the sku).** This single mismatch is the most common reason catalog ads show “no matching events” and dynamic retargeting never works.

Done when: The Meta Pixel Helper browser extension shows a green check, and the events appear in Events Manager > Test events with the correct content_ids.

D6. Add the Conversions API (server-side events)

Goal: send the same key events from your server so ad blockers, iOS privacy and flaky connections do not lose sales. Meta recommends running Pixel and Conversions API together (a “redundant setup”).

Needs from client: A Conversions API **access token** and the Pixel ID (C8). Ask the client to generate it in Events Manager and send it through a password manager.

1. Add `META_CAPI_TOKEN` to server-only environment variables (no NEXT_PUBLIC prefix).
2. Create `lib/capi.ts` below. It hashes email and phone with SHA-256 (lowercase and trim first), as Meta requires, and normalises Algerian numbers to international form (0550123456 becomes 213550123456).

3. Call it from the server code that saves the order (D7), passing the **same event id** that the browser uses.
4. Read the `_fbp` and `_fbc` cookies, the client IP (`x-forwarded-for`) and the user agent from the request, and pass them in. They greatly improve matching.
5. During testing set `META_TEST_EVENT_CODE` (from Events Manager > Test events). Remove it before going live, or the events will not count.

File: lib/capi.ts

```
// lib/capi.ts  (server only)
import crypto from "crypto";

const sha = (v?: string) =>
  v ? crypto.createHash("sha256").update(v.trim().toLowerCase()).digest("hex") : undefined;

export function normalizeDzPhone(raw?: string) {
  if (!raw) return undefined;
  let d = raw.replace(/\D/g, "");
  if (d.startsWith("0")) d = "213" + d.slice(1);   // 05xx... -> 213 5xx...
  return d;
}

type Evt = { name: string; eventId: string; url: string; email?: string; phone?: string;
  fbp?: string; fbc?: string; ip?: string; ua?: string; value?: number; currency?: string;
  contentIds?: string[]; contents?: { id: string; quantity: number }[] };

export async function sendCapiEvent(e: Evt) {
  const body: any = {
    data: [{
      event_name: e.name, event_time: Math.floor(Date.now() / 1000), event_id: e.eventId,
      event_source_url: e.url, action_source: "website",
      user_data: {
        em: e.email ? [sha(e.email)] : undefined,
        ph: e.phone ? [sha(normalizeDzPhone(e.phone))] : undefined,
        fbp: e.fbp, fbc: e.fbc, client_ip_address: e.ip, client_user_agent: e.ua,
      },
      custom_data: { value: e.value, currency: e.currency ?? "DZD", content_type: "product",
                     content_ids: e.contentIds, contents: e.contents },
    }],
  };
  if (process.env.META_TEST_EVENT_CODE) body.test_event_code = process.env.META_TEST_EVENT_CODE;

  const res = await fetch(
    `https://graph.facebook.com/v26.0/${process.env.NEXT_PUBLIC_META_PIXEL_ID}/events` +
    `?access_token=${process.env.META_CAPI_TOKEN}`,
    { method: "POST", headers: { "Content-Type": "application/json" }, body: JSON.stringify(body) });
  if (!res.ok) console.error("CAPI error", res.status, await res.text());
}
```

Note: De-duplication. Meta treats a browser event and a server event as the same when the Pixel's *event name* and *eventID* match the server's *event_name* and *event_id*; duplicates received within 48 hours are discarded. For Purchase, use the order id (for example `purchase_<orderId>`), so a refresh of the thank-you page cannot count twice. Include *event_source_url* on website events; it should be on your verified domain.

Done when: In Events Manager the Purchase event shows source **Browser and Server**, and no duplicate warning appears.

D7. Checkout and the Purchase event

1. In the server action or API route that inserts the order into Supabase, after the insert succeeds, generate `eventId = \`purchase_${order.id}\``.
2. Call `sendCapiEvent({ name: 'Purchase', eventId, ... })` with value, currency, content ids and quantities, hashed customer data, and the cookies/IP/user agent.
3. Return the same `eventId` to the client (for example in the redirect to `/thank-you?order=...`) and fire `track('Purchase', {...}, eventId)` once on that page. Guard it with a flag in `sessionStorage` so reloads do not fire again.
4. Never let a CAPI failure break the order. Wrap the call in try/catch (or use Next.js `after()`) so the customer always gets their confirmation.

Cash on delivery (common in Algeria): choose the definition with the client

Option	How	Pros	Cons
A. Purchase = order placed	Fire Purchase when the form is submitted.	Instant, plenty of signals, easy.	Includes fake or cancelled orders, so ads may learn from low-quality leads.
B. Purchase = order confirmed	Fire browser event as *InitiateCheckout* or a custom event; send server *Purchase* once the client confirms by phone.	Cleaner signal, real sales.	Delayed and fewer events; needs an admin "confirm" action. Meta rejects server events whose event time is too old (roughly 7 days; check current docs).

Note: A common compromise: send *Purchase* at order placement while the shop is new (so campaigns can learn), then move to confirmed orders once volume is healthy.

D8. Domain verification: the DNS record

Goal: prove to Meta that the shop's domain belongs to the client's business portfolio.

Needs from client: The TXT value from Business Settings (step C6), or registrar access if you add it for him.

1. Log in to the domain registrar or DNS host (Cloudflare, Namecheap, GoDaddy, and so on).
2. Open the DNS records of the **root domain** (the domain without www or https).
3. Add a record: **Type** TXT, **Host/Name** @, **Value** exactly the string Meta provided, starting with `facebook-domain-verification=`. Leave TTL on auto or default. Existing TXT records (SPF, Google) stay; multiple TXT records are fine.
4. Wait for propagation (minutes, sometimes up to 72 hours). Check with a TXT lookup tool or `dig TXT yourshop.com +short`.
5. Tell the client to click **Verify domain**. **Do not delete the record afterwards.** Meta keeps checking it.

Watch out: Paste the value without quotes or spaces. A domain can be verified by only one business portfolio at a time, so if it was previously used by an agency or an old account, that connection has to be removed first.

D9. Consent, privacy and security

- **Consent banner.** Load the Pixel and send CAPI events according to the consent you collect. A simple approach: load the Pixel only after the visitor accepts; keep server events limited to what the user agreed to. Confirm the requirements with the client (and a lawyer if needed). Algeria has a personal data protection law (Law 18-07), so have the client check what applies to his business.
- **Hash personal data** (email, phone, name) before sending; never send raw values. Do not send card data, passwords or health data.
- **Secrets.** Keep META_CAPI_TOKEN, META_FEED_TOKEN and the Supabase service key in server-side environment variables only. Rotate the CAPI token if it was ever pasted in chat or email.
- **Least privilege.** Ask for Advertiser-level or task-based access, not full Admin, unless a task truly needs it. Ask the client to enable two-factor authentication for everyone in the portfolio.
- **Privacy page.** Add a privacy policy that mentions Meta Pixel and advertising cookies. Meta can also ask for it during ad review.

D10. Testing and QA checklist

Run this together with the client before the first ad is launched.

- [ ] Feed URL returns 200 with the right token, 403 without it; all required columns present.
- [ ] Commerce Manager shows the expected item count; no blocking errors; sample items display the right image, price and link.
- [ ] Domain shows **Verified** in Business Settings > Brand Safety > Domains.

- [ ] Meta Pixel Helper: Pixel found, no errors, on home, product, checkout and thank-you pages.
- [ ] Events Manager > Test events: ViewContent, AddToCart, InitiateCheckout and Purchase appear with correct content_ids, value and currency.
- [ ] Server events appear with the test code; the same Purchase shows once (deduplicated) with source Browser and Server.
- [ ] Place a real test order on mobile: order saved in Supabase, Purchase counted once, thank-you page reload does not add another.
- [ ] Event Match Quality for Purchase is reasonable (email and phone present); fix missing parameters if it is low.
- [ ] Catalog > Diagnostics: events are matched to catalog items (content_ids match the feed ids).
- [ ] Product link opened from an ad preview works on mobile and keeps UTM parameters.
- [ ] META_TEST_EVENT_CODE is removed from production environment variables.

D11. Handover, monitoring and optional real-time sync

1. **Document** for the client: where the feed lives, how product data maps to fields, who to call if a catalog error appears.
2. **Monitoring.** Log CAPI failures and set an alert; once a week check Commerce Manager diagnostics and Events Manager for dropped events.
3. **Token expiry.** If you use a system-user or generated token, note when it expires and how to regenerate it.
4. **Optional: real-time stock sync.** The feed is enough for most shops. If stock changes quickly, keep static fields in the feed and push volatile ones (price, stock) with the Items Batch API (`POST /{catalog_id}/items_batch`) from a Supabase database webhook. Items Batch requests are asynchronous: store the returned handle and check its status. Use your own sku as the item id there, and never include the same fields in both the feed and batch updates, or they will overwrite each other.

PART 2

What the business owner does

Meta accounts, domain, catalog, access, reels and ads

Part 2: Business owner tasks

Do these in order. The developer will tell you what to copy and send. **Never send your Facebook password.** Meta lets you give access safely, as explained in C8.

C0. Before you start

- [ ] A personal Facebook profile that you control, with **two-factor authentication** turned on (Facebook > Settings > Accounts Center > Password and security > Two-factor authentication).
- [ ] A business email address you check regularly (Meta sends verification emails there).
- [ ] Access to the domain registrar login (or the name of whoever manages the domain), so the DNS record in C6 can be added.
- [ ] A payment method for ads: a credit or debit card that works for international online payments. Meta's accepted payment methods depend on the country, so confirm in billing setup (C4).
- [ ] Your business details: legal or trade name, address, phone, website, logo and a cover image.
- [ ] Your Instagram login (if you already have the shop's Instagram account).

C1. Create the Meta Business portfolio

The portfolio (formerly "Business Manager") is the container that owns your Page, Instagram, ad account, Pixel and catalog. You, the owner, must own it.

1. Go to **business.facebook.com** and log in with your personal Facebook profile.
2. Click **Create a business portfolio** (or Create account).
3. Enter the business name, your name and your business email, then submit.
4. Open the confirmation email from Meta and click the link to confirm.
5. Open **Business Settings** (gear icon, bottom left) > **Business info**, and fill in the legal name, address, phone and website. This helps avoid restrictions later.
6. Business Settings > **Security Center**: turn on two-factor authentication requirement for the portfolio if offered.
7. Write down the **Business ID** shown under Business info. You will send it to the developer if you choose partner access in C8.

> **Watch out:** Create the portfolio from **your** account, not the developer's. If the developer owns it, you can lose control of your own ad history and data.

C2. Create or connect your Facebook Page

1. If you already have a Page for the shop: Business Settings > **Accounts > Pages > Add > Add a Page**, then search for it and request access (you must be an admin of the Page).
2. If not: Business Settings > **Accounts > Pages > Add > Create a new Page**. Give it the shop name, a category (for example "Shopping & retail"), then upload the logo and cover.
3. On the Page, fill in the **About** section: contact, hours, address and the website link.
4. Write down the **Page ID** (Page > About > Page transparency, or in Business Settings > Pages).

C3. Instagram professional account

1. In the Instagram app: **Settings > Account type and tools > Switch to professional account** and choose **Business**. (Personal accounts cannot be used with Meta ads tools.)
2. In Instagram: **Edit profile > Page** and connect it to the Facebook Page from C2.
3. In Business Settings: **Accounts > Instagram accounts > Add**, log in to Instagram when asked, and confirm.
4. Put the shop link in the bio.

> **Done when:** The Page and the Instagram account both appear in Business Settings under your portfolio.

C4. Create the ad account and add billing

1. Business Settings > **Accounts > Ad accounts > Add > Create a new ad account**.
2. Enter a name, choose your **time zone** (Africa/Algiers) and **currency**. **Currency and time zone cannot be changed later**, so choose carefully, and discuss with the developer which currency the shop uses.
3. Assign yourself as admin, and assign the Page to the ad account when asked.
4. Add a payment method: Business Settings > **Payments** (or Ads Manager > Billing > Payment settings > Add payment method). Follow the prompts for your card.
5. Write down the **Ad account ID** (the number shown at the top of Ads Manager or in Business Settings).

> **Watch out:** If the card is declined, do not keep retrying. Repeated failures can restrict the account. Contact your bank to allow international online payments, or try another card.

C5. Create the Pixel (dataset)

1. Open **Events Manager** (business.facebook.com/events_manager) and select your portfolio.
2. Click **Connect data** (or the green + button) > choose **Web** > name it (for example “MyShop Pixel”) > enter your website URL.
3. When Meta asks how to install it, choose the manual or “install code yourself” option, because the developer installs it in the code. You do not need to paste anything yourself.
4. Copy the **Pixel ID** (a number, about 15 digits) from Events Manager > Settings, and send it to the developer.
5. Connect it to the ad account: Business Settings > **Data sources > Datasets / Pixels** > select the Pixel > **Add assets** > Ad accounts > select your ad account > enable **Manage** access > Save.

C6. Verify your domain

Required before you can fully use the catalog and Pixel for ads on your own site.

1. Business Settings > **Brand Safety** (“Brand safety and suitability”) > **Domains** > **Add** > **Create a new domain**.
2. Enter the **root domain only**, for example `yourshop.com` (no https://, no www, no slashes), and click Add.
3. Select the domain, then choose the verification method **Update the DNS TXT record with your domain registrar**.
4. Meta shows a code like `facebook-domain-verification=abc123...`. **Copy it exactly** and send it to the developer (or add it yourself using the next step).
5. At the domain registrar: DNS settings > add a record of type **TXT**, host `@`, value = the code. Save.
6. Wait (usually minutes; can take up to 72 hours), then return to the Domains page and click **Verify domain**.
7. **Leave the TXT record in place permanently.**

> **Note:** Alternative methods exist (uploading an HTML file or adding a meta tag to the site). The developer can use these instead of DNS if DNS access is difficult. You only need one method.

> **Done when:** The domain shows as **Verified** in the Domains list.

C7. Create the catalog and connect the feed

1. Business Settings > **Data sources > Catalogs > Add > Create a new catalog**. Choose type **E-commerce**, name it, and select your portfolio as the owner.
2. When offered, connect the catalog to your Pixel and to your Page. You can also do this later from the catalog's settings.
3. Open **Commerce Manager** (business.facebook.com/commerce) and select the catalog.
4. Go to **Catalog > Data sources** > **Add items** > **Data feed** > **Next**.
5. Choose that you want to upload using a link ("Use a URL or Google Sheets" / scheduled feed) and paste the **feed URL** from the developer. Leave username and password empty unless the developer told you otherwise.
6. Give the feed a name, set the **default currency** (the same as in the shop), and choose the schedule. Recommended: **daily** to start, **hourly** if stock or prices change a lot.
7. Click **Upload** and wait a few minutes. Then open **Catalog > Items** to see the products, and **Overview / Diagnostics** to see warnings.
8. Send the developer a screenshot of any errors; he will fix them in the website data, then you click **Upload now** in the feed settings to re-fetch.
9. Create a **product set** (Catalog > Sets > Create set) such as "All products" and, if you like, "Best sellers"; ads can promote whole sets.
10. Write down the **Catalog ID**.

> **Watch out:** Do not edit products manually in Commerce Manager. The scheduled feed will overwrite your changes. Change them in the shop's admin panel.

C8. Give the developer access (without sharing your password)

There are two safe ways. Pick one.

Option 1: Partner access (best if the developer has his own business portfolio)

1. Ask the developer for his **Business ID** (Business Settings > Business info in his portfolio).
2. In **your** Business Settings: **Users > Partners > Add > Give a partner access to your assets**.
3. Paste his Business ID and continue.
4. Select asset types one by one and grant: **Pixel / dataset** (Manage), **Catalog** (Manage), **Ad account** (Advertiser, not Admin), **Page** (task-based access, for example create ads/manage content), **Instagram account** (if he needs to see it).
5. Save. The developer then confirms he can see the Pixel in Events Manager and the catalog in Commerce Manager.

Option 2: Add him as a person

1. Business Settings > **Users > People > Add**, enter his email, and choose **Employee** access (not Admin).
2. Select the same assets as above and grant task-level permissions.
3. He accepts the invitation by email.

Generate the Conversions API token

1. Events Manager > select your Pixel > **Settings** > scroll to **Conversions API** > **Generate access token** (the label may read "Set up manually" first).
2. Copy the token and send it to the developer through a **password manager share** or another encrypted channel, not plain WhatsApp or email.
3. In the same area, open **Test events** and give the developer the **test event code** (for example TEST12345) while testing.

> **Done when:** The developer has: Pixel ID, Catalog ID, Ad account ID (optional), the CAPI token, and confirms he can see the Pixel and catalog.

C9. Create the reels (your creative work)

1. Film **vertical 9:16** video (1080 x 1920 px ideal). Short is better: usually 10-30 seconds for ads.
2. **First 3 seconds** must grab attention: show the product in use, the result, or a clear offer. Do not start with a logo.
3. Add **on-screen captions**, since many people watch without sound. Keep important text away from the very top and bottom of the screen (the app places buttons there).
4. Show: what it is, the price or deal, and how to order. End with a clear spoken or written call to action ("Order now, delivery to all wilayas", for example).
5. Use your own footage and licensed music. Avoid copyrighted songs in ads; use Meta's sound library or royalty-free music.
6. Post it normally to Instagram Reels (and Facebook Reels, if you want) from the mobile app with a short caption. Put the product link in the caption on Facebook; on Instagram use the link in bio or the ad button.
7. If your account is eligible for product tags, tag the product from the catalog on the reel. If not, you will use the ad button instead.

> **Note:** Produce several short variations (different first 3 seconds, different product angle). Meta delivers better when it can compare several creatives.

C10. Launch an ad with the "Order Now" button

Use the Ads Manager flow below. Meta may rename some options; the structure stays the same: campaign, ad set, ad.

1. Open **Ads Manager** (adsmanager.facebook.com), confirm the right ad account in the top left, and click **Create**.
2. **Campaign:** objective **Sales** (also called conversions). Give it a name such as "Sales - Reels - Nov". Choose Advantage+ campaign setup only if you want Meta to automate audiences and placements; for control, use the manual setup.
3. **Ad set > Conversion:** location **Website**. Select your **Pixel** and the optimisation event **Purchase** (or the event you agreed in D7 with the developer).
4. **Budget:** start small but not tiny, for example a daily budget you can afford for 5-7 days without touching it. Ads need time to learn; a rule of thumb is roughly 50 optimisation events per week per ad set to leave the learning phase.
5. **Audience:** country Algeria (or the wilayas you deliver to), age 18+ or as relevant, language Arabic and/or French. Start broad and let Meta optimise; add interests only if the product is very niche.
6. **Placements:** keep **Advantage+ placements** on, or select Instagram Reels and Facebook Reels if you want to focus on reels.
7. **Ad > Identity:** select your Facebook Page and Instagram account.
8. **Ad > Creative:** choose **Use existing post** (select your reel) or upload the video. For the best-performing posts, using the existing reel keeps its likes and comments.
9. **Destination:** website URL = the product page (copy it from the shop). Add UTM parameters if you want, for example `?utm_source=meta&utm_medium=reel&utm_campaign=nov_promo`.
10. **Call to action:** open the dropdown and choose **Order Now** (the button appears under the video on Facebook and Instagram).
11. Check the preview for Instagram Reels and Facebook Reels, then click **Publish**. Ads normally go through a review of up to about a day.

> **Note:** **Alternative: catalog ads.** Instead of one reel, create a **Sales** campaign using your **catalog** and a **product set**. Meta then shows the right products dynamically, including to people who viewed or added a product to the cart (this requires the Pixel with correct content_ids from D5). You can use video creatives in these formats too.

> **Watch out:** Do not edit the budget, audience or creative every day. Significant edits restart learning. Review after 3-5 days and decide with data.

C11. Weekly routine and good habits

- [ ] Monday: check Commerce Manager diagnostics, with no new catalog errors or rejected items.
- [ ] Check Ads Manager: spend, purchases, cost per purchase and the click-to-purchase rate. Compare orders in the shop admin with what Meta reports; they will not match perfectly, so look at trends.
- [ ] Turn off ads that spend with no orders after a reasonable amount of spend and time; scale the ones that work gradually (for example +20% every few days).
- [ ] Publish at least one new reel per week and promote the best performers.
- [ ] Reply to comments and messages quickly; this affects conversion and ad quality.
- [ ] Keep stock, prices and photos updated only in the shop's admin panel; the feed syncs it to Meta.
- [ ] Make sure every person with access uses two-factor authentication. Remove people who no longer work with you (Business Settings > Users).
- [ ] Tell the developer if you change products, prices, shipping or the checkout, since tracking may need updating.

Appendix A. Handover sheet

Fill this in together. Share secrets only through a password manager.

Item	Who provides	Where to find it	Secret?
Business portfolio ID	Client	Business Settings > Business info	No
Page ID	Client	Business Settings > Accounts > Pages	No
Instagram account	Client	Business Settings > Accounts > Instagram accounts	No
Ad account ID	Client	Ads Manager (top left) / Business Settings	No
Pixel ID	Client	Events Manager > Settings	No
Catalog ID	Client	Commerce Manager > Catalog settings	No
Verified domain TXT value	Client	Business Settings > Brand Safety > Domains	No (public DNS)
Conversions API token	Client	Events Manager > Settings > Conversions API	**Yes**
Test event code	Client	Events Manager > Test events	No (temporary)
Feed URL and token	Developer	Deployed site: /api/meta/feed.csv?token=...	**Yes**
Developer Business ID (if partner access)	Developer	Developer's Business Settings > Business info	No

Appendix B. Troubleshooting

Symptom	Likely cause	Fix
Items disapproved: domain not verified	TXT record missing, mistyped, or not propagated	Re-copy the value, verify at the root domain with host @, wait up to 72 hours, then click Verify again.
Items rejected for image	Image too small, not public, or blocked	Use at least 500x500 px public HTTPS images; test the URL in a private window.
Price errors	No currency, comma decimals, or zero price	Use the format *1500.00 DZD*; exclude products with price 0.
Items disappear after upload	A replace schedule deleted items missing from the feed	Check the published filter and the feed contents; it is expected for unpublished items.
Catalog shows “no matching events”	Pixel content_ids differ from feed ids	Send the sku as content_ids everywhere; test with Pixel Helper.
Pixel events not showing	Ad blocker, consent not given, wrong Pixel ID, script not loaded	Test in a clean browser; check NEXT_PUBLIC_META_PIXEL_ID; check the consent logic.
Purchases counted twice	Missing or different event ids between browser and server	Use the same event name and id (purchase_orderId) in both; guard the thank-you page.
Server events not arriving	Wrong token, token missing permissions, wrong Pixel ID	Check the response body logged by sendCapiEvent; regenerate the token.
Events visible only in Test events	Test event code still set in production	Remove META_TEST_EVENT_CODE and redeploy.
Cannot see the Pixel as developer	Access not assigned to the partner or person	Client assigns the Pixel asset (Business Settings > Datasets) to the partner or person with Manage access.

Symptom	Likely cause	Fix
“Order Now” missing in the ad	Chosen objective or format does not support the button	Use a Sales objective with a single video or image creative; select the button in the Call to action dropdown.

Appendix C. References

- Meta ad call-to-action options (Order Now listed for single image/video ads): facebook.com/business/help/1767640623555898
- Meta Catalog Batch API and Commerce catalog overview: developers.facebook.com/documentation/ads-commerce/commerce-platform/catalog/
- Conversions API: handling duplicate Pixel and server events: developers.facebook.com/documentation/ads-commerce/conversions-api/deduplicate-pixel-and-server-events
- Conversions API server event parameters: developers.facebook.com/documentation/ads-commerce/conversions-api/parameters/server-event
- Domain verification: developers.facebook.com/documentation/sharing/domain-verification/verifying-your-domain
- Instagram product tagging (for later, if eligible): developers.facebook.com/documentation/instagram-platform/instagram-api-with-facebook-login/product-tagging
- Facebook Reels Publishing API (only if you later automate posting): developers.facebook.com/documentation/video-api/guides/reels-publishing

This plan was assembled from Meta's developer documentation and help guides as of November 2026. Meta changes interface labels, limits and requirements regularly, so confirm anything critical in the live documentation.